

CS3244 HOMEWORK 3:

LABELLED FACES IN THE WILD

Team name : CRC

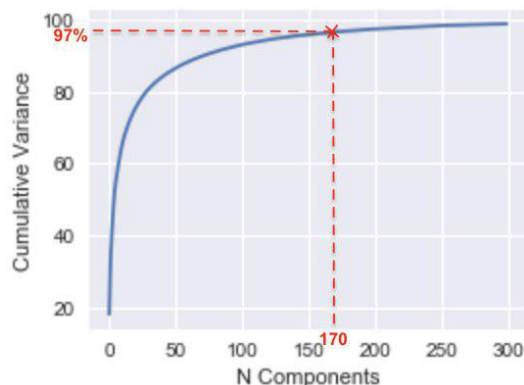
Tiong YaoCong	(A0139922Y)
Chen Rui Wen	(A0138431L)
Yim Chia Hui	(A0139379M)

1 ABSTRACT

Initially, our team aimed to try out as many learning methods as possible to identify the best one for this assignment. We explored Principal Component Analysis (PCA), Extreme Gradient Boosting (XGBoost), Support Vector Machines (SVM) and Convolutional Neural Networks (CNN). This led us to do more research on the complexity and the feasibility of each algorithm. We have grouped our attempts into three different approaches XGBoost, SVM and CNN. In this paper, we would like to present our findings on the attempts of different approaches and its results respectively.

2 PRINCIPAL COMPONENT ANALYSIS (PCA)

Before attempting using XGBoost or SVM, we explored the possibility of dimension reduction of the data of using PCA.



We think that 97% of the variance is sufficient to describe the dataset accurately, thus we have decided to keep 170 out of the 300 initial components while we reduce 43.3% insignificant dimensions of the

original dataset to speed up the classifier model processes in 3.1 and 3.2 and ignore the extreme anomalies.

3 APPROACHES TO LABELLED FACES IN THE WILD (LFW)

3.1 EXTREME GRADIENT BOOSTING (XGBoost)

XGBoost is an implementation of gradient boosted decision trees designed for speed and performance. It uses Dropouts Additive Regression Trees (DART) which prevent over-specialisation, wherein decision trees added at later iterations tend to impact the prediction of only a few instances and make insignificant effect towards the prediction of the majority of the data. Hence, we decided to attempt using XGBoost decision tree as our first approach to the LFW problem set. On our first attempt on the LFW problem set, we managed to achieve an F1 score of 50% and 66% accuracy as reported by Kaggle with the data optimised with PCA.

3.2 SUPPORT VECTOR MACHINE (SVM)

To continue to explore another different possible method for LFW problem set, we decided to try out using SVM. After reading several research papers, we chose to use radial-based function (RBF) kernel with the grid-search method to find the optimal hyperparameters available. However, we only managed to improve the F1 score accuracy to be at 81%. In the attempt of improving the result of SVM, we came upon a report where even if we were to use the best hyperparameters on the SVM through Bayesian optimisation, the best results to-date by using SVM approach is at 88%^[1].

3.3 CONVOLUTIONAL NETURAL NETWORKS (CNN)

In our final approach, using CNN is our last method model for LFW problem set. We used different library such as Tensorflow and Keras to train and data manipulation to generate more dataset for training and validation purpose and to generalise LFW problem set better.

3.3.1 Experimenting CNN Models

Training data set is randomly augmented by rotating it with a different angle, flipping horizontally and vertically. This is because the same picture should represent the same object even though if augmented slightly.

We tested CNN with different parameters as experimenting is the only way to go. In particular, we found out that increasing the number of epoch does not improve the Kaggle's score of our benchmark of 94.4%.

Epoch, dropout rate, batch size	Kaggle Score	F1 Score
50, 25%, 8	92.54%	95.54%
50, 25%, 16	94.4%	97.84%
50, 50%, 16	94.4%	98.54%
50, 25%, 32	91.3%	96.59%
500, 25%, 16	92.54%	99.36%
1000, 25%, 16	91.92%	99.9%

Table 1: Result of CNN attempt 1

We have tried to change to have an epoch. It not only has a worse prediction score but also take a longer to train too. We also make changes in the batch size and notice that smaller batch size is performing worse than higher batch size. The result could be due to the small dataset that is given to us and there are insufficient data points in each batch for the model to learn.

Seeing that all these parameters did not improve the accuracy. One of the possible reasons is that we could be overfitting the model with the given training dataset. Hence, not generalising enough for the actual test data provided by Kaggle. Thus, we look into how we can pre-process the data better.

3.3.2 Pre-process & generating dataset

After experimenting different CNN model by tweaking the values of each model. Our analysis of our current CNN model is either not generalising well enough or overfitting on the given training dataset. Hence, we used Keras and OpenFace library to do image augmentation and validity check before saving the new image as a new dataset.

We split the given training dataset into two different set, 1 for training (80%) and the other for validation

(20%) and generate an additional of 80% training (1000) and 20% validation (200) dataset for CNN model to train and generalise. Then, we were able to utilise the F1 score as a way to gauge the performance of all the specific CNN model on the validation set.

3.3.3 Improved CNN Model

To improve our result from our previous experiments, we set out to research more on the optimal Epoch, dropout rate and batch size to use for CNN model and changes to the ideal Epoch, dropout rate, and batch size based on our research result.

Epoch, dropout rate, batch size	Kaggle Score	F1 Score
40, 25%, 50	93.58%	97.2%
100, 50%, 25	95.03%	98.34%
100, 65%, 25	95.652%	92.4%
100, 75%, 25	95.652%	93.46%
100, 90%, 25	95.652%	91.08%

Table 2: Result of CNN attempt 2

After changing our CNN model and with a bigger dataset, we saw an improvement in our result. Hence, this could be that our initial analysis may be correct where the given training dataset is too small and insufficient for the model to generalised optimally.

Moreover, by increasing the dropout rate, we are able can observe another breakthrough in accuracy from 95.03% to 95.652%. However, by further increasing the dropout rate does not improve our score as reported by Kaggle. This could be that even though our new model is not overfitting the dataset as before, it still not generalising well enough for the actual dataset in Kaggle.

With our new model and generated dataset, we managed to break through our highest accuracy from the previous experiment of 94.4% to 95.652% as reported by Kaggle.

4 RESULTS & ANALYSIS

Approach	XGBoost	SVM	CNN
F1 score	50%	81%	93.46%
Kaggle	66%	-	95.652%

After experimenting different methods, the best reported Kaggle result is an accuracy of 95.652% with CNN. Hence, we concluded that using CNN is the best for LFW problem set through the different approaches we attempted.

5 POST RESULT ANALYSIS

After researching even further on how to improve our accuracy and found that one of the best attempts to-date was by Lu and Tang with an accuracy of 98.52%^[2] on the LFW dataset using Gaussian Process through the feature extraction Gaussian Process Latent Variable Model that surpass the benchmark for image recognition by humans which is at 97.53%.

However, there are insufficient time, educational resources and Python library on this model for my team to learn and replicate the classifier model.

6 CONCLUSION

In conclusion, we managed to score an accuracy of 95.652% in Kaggle in the competition and with more time or a more powerful computer, we may manage to tweak our model to generalise and learn even better.

We also learned that achieving a higher F1 score is not necessarily to do better with the actual dataset as there could be unseen circumstances and we do not want overfitting to occur in our model. By experimenting, we can tweak and update our parameters in the direction after analysis to make our model more accurate.

7 WORK ALLOCATION

We split our 3-man work allocation based on model exploration and implementation.

8 STATEMENT OF INDIVIDUAL WORK

We, A0139379M, A0139922Y, A0138431L, certify that we have followed the CS3244 Machine Learning class guidelines for homework assignments. In particular, we expressly vow that we have followed the Facebook rule in discussing with others in doing the assignment and did not take notes (digital or printed) from the discussions.

9 INCLUDED FILES

1. README.pdf – This document, detailing the entire report.
2. hw3-lfw-svm-xgboost.ipynb – Source code for XGBoost and SVM model
3. hw3-lfw.ipynb – Source code for final model of Convolutional Neural Network (Final Model)
4. hw3-lfw-improved-cnn – Source code for improved model of CNN
5. shape_predictor_68_face_landmarks.dat – Dependency for validation of pre-processing of generated data

10 REQUIRED DEPENDENCY

The following are the required dependencies that do not come in the default Anaconda package:

1. OpenCV
2. dlib
3. Keras
4. Tensorflow
5. Xgboost

11 REFERENCES

- [1] University of Massachusetts Amherst. (n.d.). Retrieved from Labeled Faces in the Wild: <http://vis-www.cs.umass.edu/lfw/results.html>
- [2] Lu, C., & Xiaoou, T. (n.d.). *Surpassing Human-Level Face Verification Performance on LFW with GaussianFace*. Retrieved from <https://arxiv.org/pdf/1404.3840v2.pdf> GaussianFace Model
- [3] EliteDataScience. (9 February, 2017). Retrieved from Keras Tutorial: The Ultimate Beginner's Guide to Deep Learning in Python: <https://elitedatascience.com/keras-tutorial-deep-learning-in-python>
- [4] Geitgey, A. (n.d.). Retrieved from Machine Learning is Fun! Part 3: Deep Learning and Convolutional Neural Networks: <https://medium.com/@ageitgey/machine-learning-is-fun-part-3-deep-learning-and-convolutional-neural-networks-f40359318721>